

Day 20 — Cloud Platforms and Enterprise AI Architecture

5-line beginner summary

1. Cloud platforms provide on-demand servers, storage, databases, networking and managed AI services without requiring companies to own physical infrastructure.
 2. AI applications use cloud compute for training and inference, object storage for datasets and models, databases for application data and Kubernetes for container orchestration.
 3. Containers package an AI application consistently, while Kubernetes deploys, scales, restarts and manages those containers.
 4. Enterprise cloud architecture must include security, secrets management, monitoring, high availability, scalability and cost controls.
 5. Databricks, RAG and MLOps use cloud infrastructure together to process data, train models, deploy APIs and continuously monitor production systems.
-

1. Why cloud is important for AI systems

AI workloads have different infrastructure requirements at different stages:

- Data preparation may require large distributed compute clusters.
- Model training may require expensive GPUs.
- Model inference may require low-latency APIs.
- RAG may require object storage, vector search, databases and LLM endpoints.
- MLOps requires pipelines, model registries, monitoring and deployment infrastructure.

Buying enough physical infrastructure for the maximum possible demand is expensive. Cloud platforms allow organizations to provision infrastructure when needed, scale it up or down and use managed services instead of operating every component themselves.

For example, a company may use:

- Small CPU machines for development.
- A GPU machine for two hours to fine-tune a model.
- A managed endpoint for production inference.
- Object storage for millions of documents.
- Kubernetes for running the API and retrieval services.

This is generally called **on-demand resource provisioning**.

Main cloud benefits for AI

Elasticity

Resources can increase or decrease according to demand.

Example:

Normal traffic:	2 API containers
High traffic:	10 API containers
Night-time traffic:	1 API container

Managed services

The cloud provider operates infrastructure such as:

- Databases
- Kubernetes control planes
- Object storage
- AI model endpoints
- Monitoring services
- Secret stores

The development team can focus more on the application and less on managing servers.

Access to specialised hardware

Cloud platforms provide CPUs, GPUs and AI accelerators. Teams can temporarily provision expensive hardware instead of buying it permanently.

Faster experimentation

A data scientist can create an environment, run an experiment and delete the environment afterward.

Global deployment

Applications can be deployed closer to users through multiple cloud regions.

Reliability and backup

Cloud services provide capabilities such as:

- Multiple availability zones
- Automated backups
- Replication
- Load balancing
- Disaster recovery

Cloud does not automatically make an application reliable, however. The architecture must still be designed correctly.

2. IBM Cloud, AWS, Azure and GCP at a high level

All four major cloud platforms provide the same fundamental building blocks:

Compute
Storage
Databases
Networking
Identity and security
Containers and Kubernetes
AI/ML services
Monitoring
DevOps services

The service names differ, but the architectural concepts are similar.

Requirement	IBM Cloud	AWS	Microsoft Azure	Google Cloud
Virtual machines	Virtual Servers for VPC	EC2	Azure Virtual Machines	Compute Engine
Object storage	IBM Cloud Object Storage	Amazon S3	Azure Blob Storage	Cloud Storage
Managed Kubernetes	IBM Cloud Kubernetes Service	Amazon EKS	Azure Kubernetes Service	Google Kubernetes Engine
Container registry	IBM Cloud Container Registry	Amazon ECR	Azure Container Registry	Artifact Registry
Managed AI platform	watsonx.ai	SageMaker AI	Microsoft Foundry/ Azure ML	Vertex AI-related services and the newer Gemini Enterprise Agent Platform naming
Secrets	IBM Cloud Secrets Manager	AWS Secrets Manager	Azure Key Vault	Secret Manager
Monitoring	IBM Cloud Monitoring	CloudWatch	Azure Monitor/ Application Insights	Cloud Monitoring
Identity	IBM Cloud IAM	AWS IAM	Microsoft Entra ID/ Azure RBAC	Cloud IAM

IBM positions **watsonx.ai** as an enterprise studio for training, validating, tuning and deploying models, including RAG development. AWS provides SageMaker AI for training and model serving. Microsoft's current Foundry architecture connects AI projects with services such as Storage, Key Vault and AI Search. Google's current documentation is evolving toward the name **Gemini Enterprise Agent Platform** for some workflows previously associated with Vertex AI. ([IBM](#))

Simple way to remember the platforms

IBM Cloud	-> strong hybrid-cloud, enterprise governance and watsonx ecosystem
AWS	-> broad cloud ecosystem and mature infrastructure services
Azure	-> strong Microsoft enterprise and identity integration
GCP	-> strong data, analytics, Kubernetes and AI ecosystem

This does not mean that one cloud is always better. Enterprises usually select a cloud based on:

- Existing technology
- Customer requirements
- Regulations
- Data location
- Employee skills

- Commercial agreements
 - Required AI models and services
-

3. Compute

Compute means the processing resources that run applications and workloads.

The main compute choices are:

3.1 Virtual machines

A virtual machine is a software-based server.

You choose:

- CPU
- Memory
- Disk
- Operating system
- GPU, when required

Example:

```
VM:
  Operating system: Linux
  CPU: 8 cores
  RAM: 32 GB
  GPU: 1 NVIDIA GPU
  Application: Model inference API
```

Advantages

- High level of control
- Custom software installation
- Suitable for legacy applications
- Useful for specialised model-serving requirements

Limitations

- The team must patch and maintain the operating system.
 - Scaling requires additional configuration.
 - Poor utilisation can increase cost.
-

3.2 Containers

Containers package the application, dependencies and runtime together.

A model API container may contain:

```
Python
FastAPI
Model-serving code
Required libraries
```

Model configuration Startup command
--

Containers are lighter than virtual machines because they share the host operating system kernel.

3.3 Serverless compute

With serverless compute, the provider manages the underlying servers.

You normally pay according to requests, execution time or resource consumption.

It is useful for:

- Event-driven processing
- Lightweight APIs
- Document ingestion
- Scheduled jobs
- Irregular workloads

Serverless model endpoints may scale automatically, but they can introduce cold-start latency. AWS, for example, documents serverless inference as suitable for workloads with idle periods that can tolerate cold starts. ([AWS Documentation](#))

3.4 GPU compute

GPUs are commonly used for:

- Deep-learning training
- LLM fine-tuning
- Embedding generation
- Large-model inference
- Image and video processing

GPUs are much more expensive than standard CPU compute. Therefore:

Use CPU when CPU is sufficient. Use GPU only when the workload benefits from it. Stop GPU resources when they are not being used.

Easy example

An insurance-document RAG system might use:

Document ingestion	-> CPU compute
Embedding generation	-> CPU or GPU
Vector indexing	-> CPU and memory
LLM generation	-> Managed model endpoint
FastAPI application	-> Container compute
Batch re-indexing	-> Temporary scheduled compute

4. Storage

Cloud applications use several types of storage.

4.1 Block storage

Block storage behaves like a disk attached to a virtual machine.

Use it for:

- Operating-system disks
- Database volumes
- Application file systems
- High-performance disk access

Example:

```
Virtual Machine
  |
  +---- Block Disk
        |
        +---- operating system
        +---- local model files
```

4.2 File storage

File storage provides shared folders accessible by multiple systems.

Use it when applications require:

- Shared directories
- Traditional file-system access
- Common model files
- Shared application assets

4.3 Object storage

Object storage stores data as objects inside buckets or containers.

Common examples include:

- IBM Cloud Object Storage
- Amazon S3
- Azure Blob Storage
- Google Cloud Storage

Object storage is widely used for AI because it can hold:

- Raw datasets
- PDFs and enterprise documents
- Images and videos
- Model files
- MLflow artifacts
- Training output

- Logs and backups
 - Delta Lake files
-

5. Managed databases

A managed database is operated by the cloud provider.

The provider normally handles much of the work related to:

- Provisioning
- Patching
- Backups
- Replication
- Failover
- Monitoring

The application team still remains responsible for:

- Schema design
- Queries
- Access permissions
- Data classification
- Retention
- Application-level security

Database choices in an AI system

Relational database

Use a relational database for structured, transactional data.

Examples:

```
Users
Permissions
Model deployment records
Feedback
API request metadata
Billing information
```

NoSQL document database

Use a document database when the records have flexible JSON-like structures.

Examples:

```
Chat sessions
Agent state
Conversation history
Application configuration
Document metadata
```

Key-value database

Use a key-value database for:

- Caching
- Sessions
- Rate-limit counters
- Temporary state

Vector database or vector-search service

Use vector search for:

- Document embeddings
- Semantic search
- RAG retrieval
- Similarity matching

Time-series or monitoring database

Use it for:

- Latency metrics
 - CPU and GPU utilisation
 - Request counts
 - Error rates
 - Model-quality metrics
-

6. Object storage in AI architecture

Object storage is especially important because AI systems process large amounts of unstructured data.

Example bucket structure

```
enterprise-ai-data/  
|  
+-- raw-documents/  
|   +-- employee-policy.pdf  
|   +-- travel-policy.pdf  
|  
+-- processed-documents/  
|   +-- extracted-policy.json  
|  
+-- embeddings/  
|   +-- embedding-batch-001.parquet  
|  
+-- models/  
|   +-- churn-model-v3/  
|  
+-- evaluation/  
|   +-- rag-golden-dataset.json  
|
```



```
+-- logs/  
  +-- inference-logs/
```

Object storage versus database

Object storage	Database
Stores files and large objects	Stores queryable records
Suitable for PDFs, images and models	Suitable for users, transactions and metadata
Usually inexpensive for large data	More expensive but supports advanced queries
Accessed using object keys	Accessed using SQL or database queries
Often forms the base of a data lake	Often supports operational applications

A common design is:

```
PDF file          -> Object storage  
PDF metadata      -> SQL/NoSQL database  
Document embeddings -> Vector database  
User permissions  -> Relational database
```

Databricks commonly processes data stored in cloud object storage, and its documentation supports workspaces across AWS, Azure and Google Cloud. ([Databricks Documentation](#))

7. Containers

A container creates a repeatable environment for an application.

Without a container:

```
Developer machine:  
Python 3.12  
Library version A  
  
Production server:  
Python 3.10  
Library version B  
  
Result:  
Application may fail.
```

With a container:

```
The same container image runs in:  
- Development  
- Testing  
- Staging  
- Production
```

Example model API container

```
FROM python-runtime

COPY requirements.txt
INSTALL dependencies

COPY application/
COPY model/

START fastapi-server
```

This is conceptual pseudocode rather than a complete Dockerfile.

Typical container workflow

```
Source code
  |
  v
Build container image
  |
  v
Run security scan
  |
  v
Push to container registry
  |
  v
Deploy container
```

8. Kubernetes basics

Kubernetes is a platform for orchestrating containers.

A container by itself can run an application, but Kubernetes provides production management capabilities.

Important Kubernetes concepts

Cluster

A group of machines on which containers run.

Node

A worker machine inside the cluster.

Pod

The smallest deployable Kubernetes unit. A pod normally contains one main application container.

Deployment

Defines the desired number of application replicas and how application updates should happen.

Service

Provides a stable network address for pods.

Ingress or gateway

Accepts external traffic and routes it to services.

ConfigMap

Stores non-sensitive configuration.

Secret

Stores sensitive configuration, although external cloud secret managers are generally preferred for enterprise secrets.

Horizontal Pod Autoscaler

Changes the number of pods according to metrics such as CPU utilisation or request demand.

Example

```
Deployment says:
"Run 3 copies of the RAG API."

Kubernetes creates:

Pod 1 -> RAG API container
Pod 2 -> RAG API container
Pod 3 -> RAG API container
```

If Pod 2 crashes, Kubernetes creates a replacement pod.

Managed Kubernetes services include IBM Cloud Kubernetes Service, Amazon EKS, Azure Kubernetes Service and Google Kubernetes Engine. IBM's managed service includes capabilities such as scheduling, self-healing, horizontal scaling, service discovery, load balancing and automated rollouts. ([IBM Cloud](#))

Containers versus Kubernetes

Containers	Kubernetes
Package and run an application	Manages many containers
Provide runtime consistency	Provides scaling and recovery
Can run on one machine	Usually manages a cluster
Do not automatically provide HA	Can distribute replicas
Built as container images	Deploys images as pods

Memory aid:

Docker/container	= package
Kubernetes	= manage

9. Model deployment on cloud

Model deployment means making a trained model available for predictions.

Common deployment options

Managed model endpoint

The cloud provider manages much of the serving infrastructure.

Model
Register model
Create endpoint
Send prediction request

Suitable when:

- The team wants fast deployment.
- Standard model-serving features are sufficient.
- Autoscaling and monitoring are required.
- Infrastructure management should be minimised.

Container deployed on Kubernetes

The team packages the model and API into a container.

Suitable when:

- Custom inference logic is required.
- Multiple services must work together.
- Special libraries are needed.
- Portability is important.
- The organisation already operates Kubernetes.

Serverless endpoint

Suitable for:

- Low or unpredictable traffic
- Event-driven inference
- Proofs of concept
- Workloads tolerant of cold starts

Batch inference

Used when immediate responses are not required.

Example:

```
Every night:
Read 10 million customers
    |
Run churn predictions
    |
Write results to database
```

Real-time inference

Used when a response is required immediately.

Example:

```
Customer transaction
    |
Fraud-detection API
    |
Prediction within milliseconds
```

Google documents model deployment as associating compute resources with a model endpoint for online inference, while AWS recommends multiple production endpoint instances to improve resilience against instance or availability-zone failures. ([Google Cloud Documentation](#))

10. Secure networking basics

Enterprise AI services should not expose every component directly to the internet.

Important networking concepts

Virtual private cloud

A logically isolated network inside the cloud.

IBM calls it a VPC, AWS calls it a VPC, Azure commonly uses VNet and Google Cloud uses VPC.

Subnet

A smaller network division inside the private cloud network.

A common design is:

```
Public subnet:
- Load balancer
- API gateway

Private application subnet:
- RAG API
```

- Model service

Private data subnet:

- Database
- Vector database

Firewall or security group

Controls permitted inbound and outbound traffic.

Example:

Internet -> Load balancer: Allow HTTPS port 443
Internet -> Database: Deny
RAG API -> Vector DB: Allow required private port

Private endpoint

Allows services such as storage, databases and model endpoints to be accessed through private networking rather than the public internet.

API gateway

Provides a controlled entry point for APIs and may handle:

- Authentication
- Rate limiting
- Routing
- Request validation
- Logging
- TLS termination

TLS

Encrypts information while it moves across the network.

Identity and access management

Controls which users, applications and services may access cloud resources.

Zero-trust principle

Do not automatically trust traffic just because it is inside the organisation's network.

Verify:

Who is calling?
What are they allowed to do?
Which resource are they accessing?
Is the request encrypted?
Should this access be logged?

IBM Cloud Kubernetes Service applies security groups and network rules to restrict cluster traffic, while Microsoft's Foundry network-isolation guidance uses private networking for

dependent services such as storage, Key Vault, container registry and monitoring. ([IBM Cloud](#))

11. Secrets management

A secret is sensitive information such as:

- Database password
- API key
- OAuth token
- Private certificate
- LLM provider credential
- Service-account credential

Incorrect approach

```
DATABASE_PASSWORD = "production-password"
LLM_API_KEY = "secret-key"
```

Problems:

- The secret may be committed to Git.
- Developers may copy it.
- Logs may expose it.
- Rotation becomes difficult.

Better approach

```
Application starts
|
Authenticates using workload identity
|
Requests secret from cloud secret manager
|
Secret manager checks permission
|
Application receives authorised secret
```

Cloud secret managers include IBM Cloud Secrets Manager, AWS Secrets Manager, Azure Key Vault and Google Cloud Secret Manager. These services are intended for credentials, API keys, tokens, certificates and similar sensitive information. ([AWS Documentation](#))

Best practices

- Never place secrets directly in source code.
- Use separate secrets for development and production.
- Rotate secrets regularly.
- Grant access only to required applications.
- Use workload identity instead of permanent credentials where possible.
- Audit secret access.

- Avoid printing secrets in logs.
 - Revoke secrets immediately after suspected exposure.
-

12. Scalability

Scalability means the system can handle increasing workload.

Vertical scaling

Increase the size of one machine.

Before: 4 CPU, 16 GB RAM After: 16 CPU, 64 GB RAM
--

Advantages:

- Simple
- May require little architectural change

Limitations:

- There is a maximum machine size.
- It may create a single point of failure.
- Upgrades may require downtime.

Horizontal scaling

Add more machines or application replicas.

Before: 1 API instance After: 8 API instances behind a load balancer

Advantages:

- Better fault tolerance
- Supports larger traffic volumes
- Works well with stateless APIs

Limitations:

- Requires distributed-system design.
- Shared state must be stored externally.
- Concurrency and consistency must be handled.

Scaling in AI applications

Different components scale differently:

API layer	-> Scale by request count
Embedding workers	-> Scale by queue size
Vector database	-> Scale by data and query load
LLM endpoint	-> Scale by tokens and concurrent requests
Data pipeline	-> Scale by records or partitions

Stateless service

A stateless API does not keep important user state inside its local memory.

Instead:

Session state	-> Redis or database
Documents	-> Object storage
Conversation state	-> Database
Model artifacts	-> Model registry/object storage

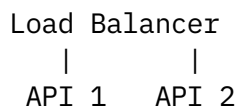
Stateless services are easier to scale horizontally.

13. High availability

High availability means the service remains accessible even when part of the system fails.

Key techniques

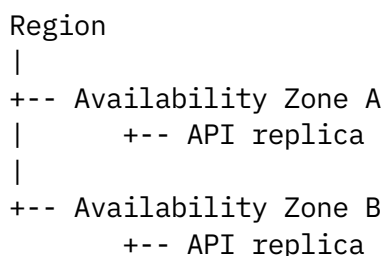
Multiple application replicas



If API 1 fails, traffic goes to API 2.

Multiple availability zones

Deploy replicas in separate data-centre zones.



Database replication

Maintain database replicas so another node can take over.

Health checks

The load balancer sends traffic only to healthy instances.

Automated restart

Kubernetes restarts failed pods.

Backup and recovery

Data should be backed up and tested for restoration.

Disaster recovery

A second region may be used for critical systems.

High availability versus disaster recovery

High availability	Disaster recovery
Handles normal component failures	Handles major regional or system failure
Usually automated	May require a recovery process
Often operates within one region	Frequently involves another region
Target is minimal interruption	Target is restoring business service

RTO and RPO

Recovery Time Objective

How quickly the service must be restored.

Example:

RT0 = 30 minutes

Recovery Point Objective

How much data loss is acceptable.

Example:

RPO = 5 minutes

14. Cost considerations

Cloud is flexible, but poorly managed cloud infrastructure can become expensive.

Major AI cost drivers

- GPU hours
- Large CPU clusters
- LLM input and output tokens
- Embedding generation
- Databricks compute
- Vector database capacity
- Object-storage volume

- Data transfer
- Database replicas
- Logging volume
- Unused development environments

IBM's watsonx.ai pricing, for example, includes token-based pricing for foundation models and hourly options for hosted deployments, illustrating the need to consider both usage-based and provisioned-resource costs. ([IBM](#))

Cost optimisation techniques

Right-size resources

Do not use a GPU when a CPU is sufficient.

Autoscale

Increase resources only when demand rises.

Shut down idle resources

Development clusters should not run continuously without reason.

Use batch processing

Batch low-priority requests rather than serving all workloads in real time.

Cache responses

Repeated questions may be answered using a validated cache.

Limit RAG context

Sending 20 irrelevant chunks to an LLM increases token cost and can reduce quality.

Use smaller models when appropriate

A small model may be sufficient for:

- Classification
- Routing
- Entity extraction
- Reranking
- Summarisation of short content

Use lifecycle policies

Move older objects to cheaper storage tiers.

Monitor cost by team and project

Use labels or tags such as:

```
project = employee-rag
environment = production
owner = ai-platform-team
cost-center = finance
```

Important cost equation

```
Total AI Cost =  
Compute  
+ Storage  
+ Database  
+ Network  
+ Model/API usage  
+ Monitoring  
+ Operational effort
```

The cheapest individual service is not necessarily the cheapest overall architecture.

15. How cloud supports Databricks

Databricks can run on AWS, Azure and Google Cloud, with cloud-specific integrations. It provides compute for data engineering, data science, machine learning and analytics. Current Databricks documentation describes both automatically managed serverless compute and customer-configurable classic compute. ([Databricks Documentation](#))

Typical architecture

```
Enterprise Sources  
  |  
  v  
Cloud Object Storage  
  |  
  v  
Databricks  
- Bronze  
- Silver  
- Gold  
- Feature engineering  
- Model training  
  |  
  v  
MLflow Model Registry  
  |  
  v  
Cloud Model Endpoint / Kubernetes
```

Cloud provides Databricks with

- Object storage
- Virtual networking
- Identity management
- Encryption keys
- Compute instances
- GPUs
- Logging
- Private connectivity

- Managed databases

Databricks documentation describes a separation between its control plane and compute plane, with classic compute resources running within the customer's cloud environment. ([Databricks Documentation](#))

16. How cloud supports RAG

A production RAG system requires more than an LLM.

RAG cloud components

Documents	-> Object storage
Extraction	-> Batch compute
Chunking	-> Data-processing job
Embeddings	-> Embedding endpoint
Vector index	-> Vector database/search service
Metadata	-> SQL/NoSQL database
LLM	-> Managed or self-hosted endpoint
RAG API	-> Containers/Kubernetes
Secrets	-> Secret manager
Monitoring	-> Cloud monitoring platform

Easy example: employee policy assistant

Ingestion path

```
HR uploads policy PDF
|
Object storage receives document
|
Event triggers ingestion job
|
Text is extracted
|
Document is divided into chunks
|
Embeddings are generated
|
Chunks and vectors are indexed
```

Query path

```
Employee asks question
|
API authenticates employee
|
Retriever searches permitted documents
|
Relevant chunks are reranked
|
Prompt is sent to LLM
```

|
Answer and citations are returned

IBM explicitly includes RAG development in watsonx.ai, while Microsoft Foundry can connect to Azure AI Search, Storage and other dependent services for retrieval-oriented applications. ([IBM](#))

17. How cloud supports MLOps

MLOps connects development, deployment and monitoring.

Cloud-based MLOps lifecycle

```
Code Repository
  |
  v
CI Pipeline
- Unit tests
- Data tests
- Security scan
  |
  v
Training Job
  |
  v
Experiment Tracking
  |
  v
Model Registry
  |
  v
Approval
  |
  v
Deployment
  |
  v
Monitoring
  |
  v
Retraining
```

Cloud services provide

- Managed training jobs
- GPUs
- Experiment tracking
- Model registries
- Container registries
- CI/CD pipelines
- Managed endpoints

- Monitoring and alerting
- Central logs
- Identity management
- Audit records

Model and application monitoring

Monitor both technical and model-level behaviour.

Infrastructure metrics

```
CPU utilisation
GPU utilisation
Memory
Network
Container restarts
Disk usage
```

API metrics

```
Request count
Error rate
Latency
Timeout rate
Rate-limit events
```

Model metrics

```
Prediction accuracy
Data drift
Concept drift
Confidence distribution
Bias indicators
```

GenAI/RAG metrics

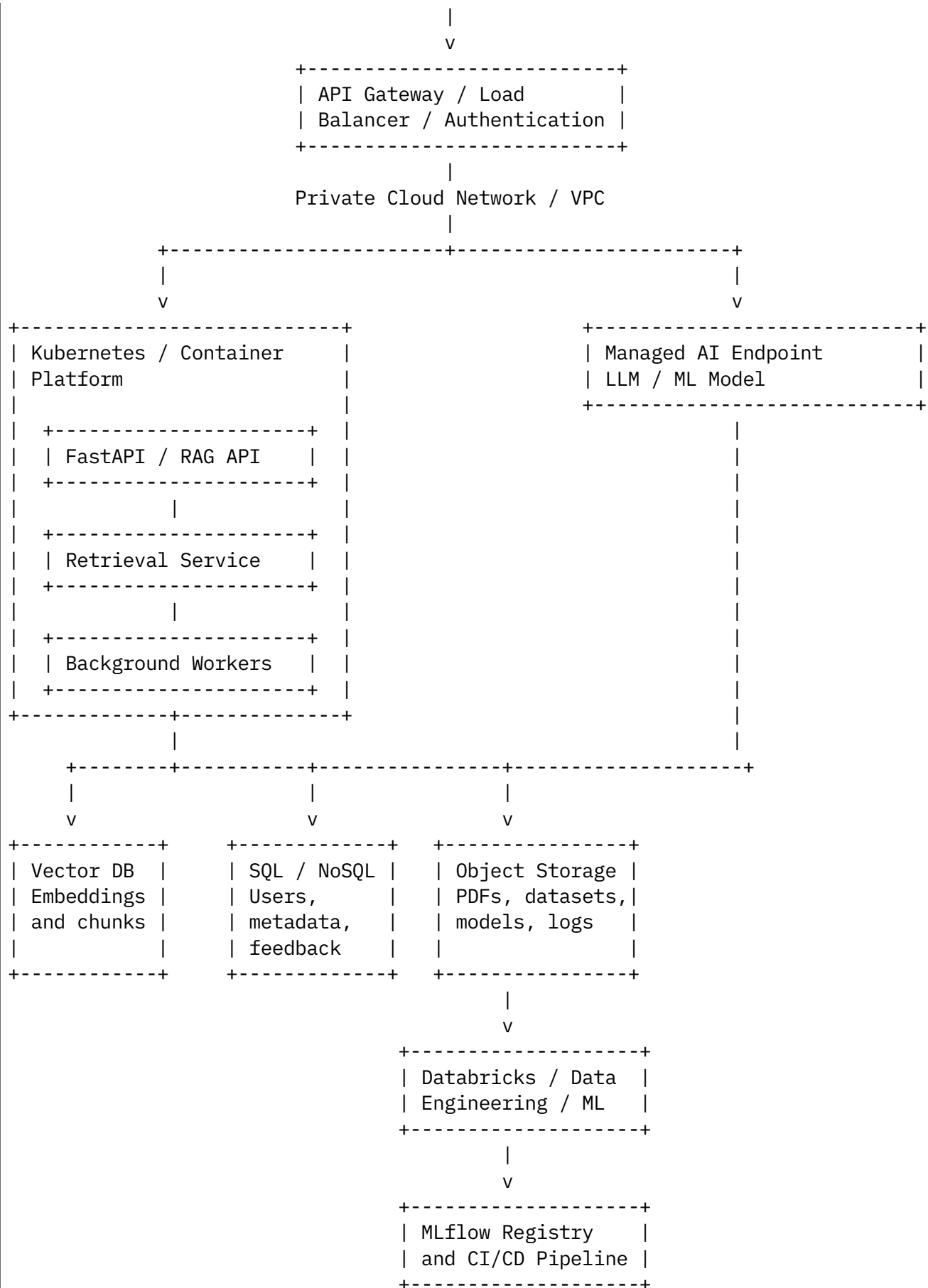
```
Retrieval relevance
Groundedness
Hallucination rate
Context precision
Context recall
User feedback
Token consumption
```

Complete cloud AI architecture diagram

```

INTERNET / ENTERPRISE USERS
|
v
+-----+
| DNS + Web Application |
+-----+

```



Cross-cutting services:

IAM | Secrets Manager | Private Networking | Encryption

Easy end-to-end example

Consider a bank developing a customer-support RAG assistant.

Development

1. Developers create the FastAPI application.
2. Data engineers ingest policy documents using Databricks.
3. Documents are stored in object storage.
4. Chunks and embeddings are indexed in a vector database.
5. The application uses an enterprise LLM endpoint.

Packaging

1. The API code is packaged in a container.
2. The container image is scanned.
3. The image is pushed to a private registry.

Deployment

1. Kubernetes deploys three API replicas.
2. An API gateway authenticates users.
3. Secrets are retrieved from a cloud secret manager.
4. Private endpoints connect the API to storage and databases.
5. The load balancer routes requests to healthy pods.

Operation

1. Autoscaling adds pods during busy periods.
2. Monitoring records latency and errors.
3. RAG evaluation measures groundedness.
4. User feedback is stored.
5. Poor results are added to the evaluation dataset.
6. A new version is tested and gradually deployed.

Pseudocode for cloud deployment workflow

```
FUNCTION deploy_ai_application():  
  
    # 1. Validate source code  
    checkout_source_code()  
    run_unit_tests()  
    run_api_tests()  
    run_model_tests()  
    run_security_checks()
```

```

IF any_test_failed:
    stop_pipeline()
    notify_development_team()
    RETURN FAILURE

# 2. Build application
container_image = build_container(
    application_code,
    model_serving_code,
    dependencies
)

# 3. Scan and publish image
vulnerabilities = scan_container(container_image)

IF vulnerabilities contain critical_issue:
    stop_pipeline()
    RETURN FAILURE

image_version = generate_version()
push_to_private_registry(container_image, image_version)

# 4. Prepare cloud infrastructure
network = ensure_private_network_exists()
cluster = ensure_kubernetes_cluster_exists(network)
secret_store = ensure_secret_manager_exists()
monitoring = ensure_monitoring_exists()

# 5. Configure secrets
store_secret(
    secret_store,
    name = "VECTOR_DATABASE_CREDENTIAL",
    value = retrieve_securely_from_approved_source()
)

# 6. Deploy to staging
deploy_to_kubernetes(
    cluster = cluster,
    environment = "staging",
    image = image_version,
    replicas = 2,
    cpu_limit = configured_cpu,
    memory_limit = configured_memory,
    secrets_from = secret_store
)

# 7. Verify staging
run_health_check()
run_smoke_test()
run_security_test()
run_model_quality_test()
run_rag_evaluation()

IF staging_validation_failed:

```

```

        rollback_staging()
        notify_team()
        RETURN FAILURE

# 8. Approve production release
request_production_approval()

IF approval_not_received:
    RETURN WAITING_FOR_APPROVAL

# 9. Deploy gradually
deploy_canary(
    environment = "production",
    image = image_version,
    traffic_percentage = 10
)

monitor(
    error_rate,
    latency,
    resource_usage,
    model_quality,
    hallucination_rate
)

IF canary_metrics_are_bad:
    rollback_to_previous_version()
    notify_operations_team()
    RETURN FAILURE

# 10. Complete rollout
increase_traffic_to_new_version(100)
enable_autoscaling()
register_deployment_metadata()
create_audit_record()

RETURN SUCCESS

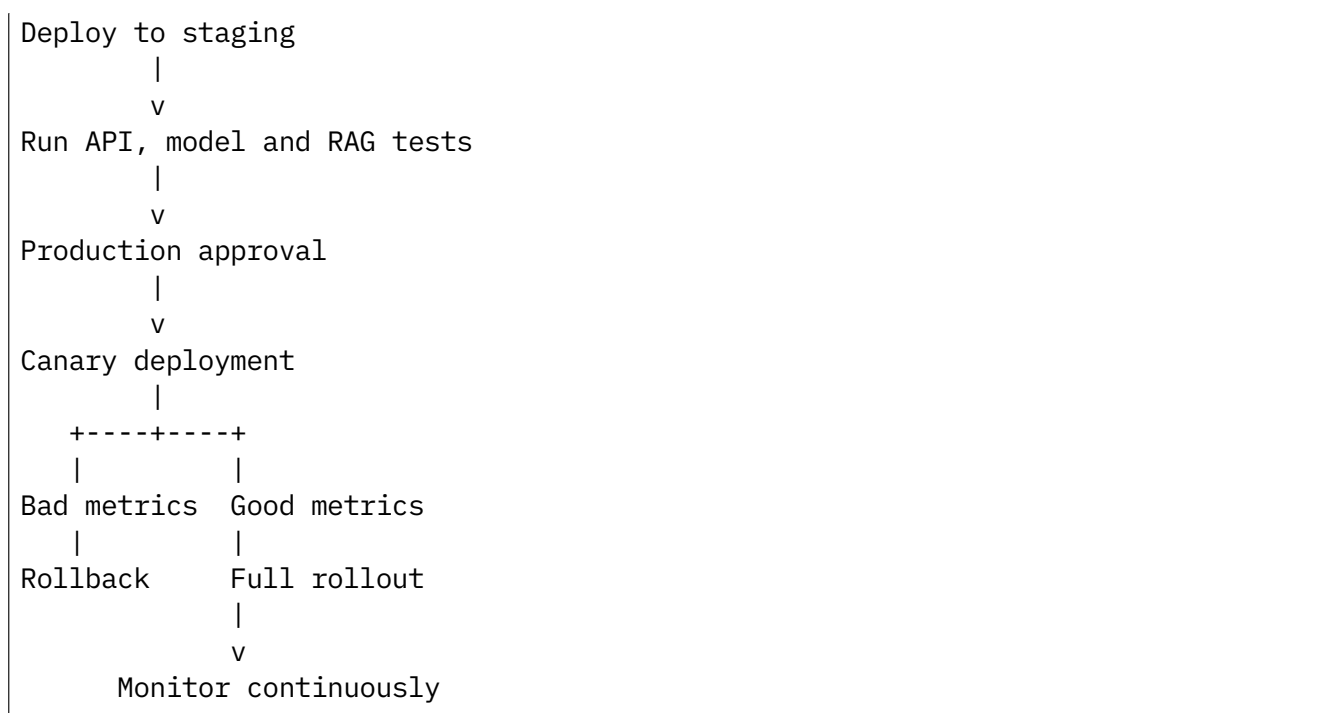
```

Cloud deployment workflow diagram

```

Developer pushes code
|
v
CI tests and security scans
|
v
Build container image
|
v
Push image to registry
|
v

```



Important architecture comparisons

VM versus container versus managed endpoint

Option	Best suited for	Management effort
Virtual machine	Maximum control and legacy workloads	High
Container service	Standard containerised applications	Medium
Kubernetes	Complex, scalable multi-service platforms	High
Managed AI endpoint	Standard model training and serving	Low to medium
Serverless endpoint	Intermittent or unpredictable traffic	Low

Scalability versus high availability

Scalability	High availability
Handles more workload	Handles failures
Adds resources	Adds redundancy
Focuses on capacity	Focuses on continuity
Example: 2 to 10 pods	Example: pods in two zones

A system can be scalable without being highly available.

Example:

Ten pods running on one node

This handles more traffic, but all ten pods may fail if that node fails.

Enterprise production checklist

Before deploying an AI system, verify:

Application

- API inputs are validated.
- Errors are handled safely.
- Health-check endpoints exist.
- The application is stateless where practical.
- Timeouts and retries are configured.

Model

- The model is versioned.
- Evaluation results are recorded.
- Rollback is possible.
- Resource requirements are understood.
- Responsible-AI checks are complete.

Security

- Authentication and authorisation are enabled.
- Secrets are stored externally.
- Least-privilege access is applied.
- Traffic is encrypted.
- Databases are not publicly exposed.
- Sensitive data is removed from logs.

Reliability

- Multiple replicas are deployed.
- Health checks are configured.
- Backups are tested.
- Availability-zone failures are considered.
- Disaster-recovery requirements are documented.

Scalability

- Autoscaling policies exist.
- Load testing is complete.
- Database and vector-store capacity is tested.
- LLM concurrency limits are understood.

Monitoring

- Logs are centralised.

- Metrics and traces are collected.
- Alerts are actionable.
- Model and RAG quality are monitored.
- Token consumption and cost are tracked.

Governance

- Model ownership is recorded.
 - Data lineage is available.
 - Production approval is documented.
 - Audit logs are retained.
 - Access is reviewed regularly.
-

Common mistakes

1. Treating cloud as only virtual machines

Cloud architecture also includes managed storage, databases, identity, monitoring, AI services and networking.

2. Using Kubernetes for every application

A small API may work better on a managed container or serverless platform. Kubernetes adds operational complexity.

3. Storing secrets in code

API keys and passwords must be stored in a secret-management service.

4. Exposing databases publicly

Databases and vector stores should generally be placed behind private networking and strict access controls.

5. Assuming autoscaling solves every problem

An API may scale while its database, vector index or external LLM endpoint becomes the bottleneck.

6. Keeping application state inside containers

Containers may restart at any time. Important state should be kept in an external database, cache or object store.

7. Running one production replica

One container, node or availability zone creates a single point of failure.

8. Ignoring GPU cost

GPU machines left running without active workloads can create significant unnecessary expense.

9. Sending excessive RAG context

More chunks increase token cost and may reduce answer quality. Retrieval and reranking should select only useful context.

10. Collecting too many logs

Excessive logs increase cost and may expose sensitive data.

11. Monitoring only infrastructure

A healthy CPU does not mean the model is producing accurate or grounded responses.

12. Using the same environment for development and production

Separate accounts, projects, subscriptions or resource groups should be used to reduce accidental production changes.

13. Not planning rollback

Every model and application deployment should have a tested rollback procedure.

14. Confusing backup with high availability

A backup helps restore data later. It does not keep the service online during an active failure.

15. Selecting services only by name

The important skill is understanding the underlying architecture:

Compute + Storage + Database + Network + Security
+ Model serving + Monitoring + Governance

Service names can change, but these concepts remain consistent.

Final memory aid

Remember enterprise cloud AI architecture as **C-S-D-N-S-R-O-G**:

C -> Compute
S -> Storage
D -> Databases
N -> Networking
S -> Security and Secrets
R -> Reliability and Scalability
O -> Observability and Operations
G -> Governance

A production AI solution is not simply:

Model + API

It is:

Data

+ Model

+ API

+ Cloud infrastructure

+ Security

+ Deployment automation

+ Monitoring

+ Reliability

+ Governance

+ Cost control